Now imagine this scenario in production for a table with hundreds of millions of rows, and thousands of queries per second trying to read from this table. This would be a disaster and cause huge amounts of errors, so it's something you'll want to avoid!

What steps can be put in place to help detect and avoid this type of scenario?

Learning To Use CONCURRENTLY By Default

PostgreSQL provides the `CONCURRENTLY` keyword as an option for a number of operations. Develop a mindset where CONCURRENTLY becomes a default option for you anytime you're working with a database connection to a Rails application that's running. Note that CONCURRENTLY is *NOT* added by default, and that there are scenarios where it's better to skip it. You'll learn some of the places where you can use it, and where it's safe to skip. It's up to you to put it into use as your default way of working if you aren't doing that already.

When `CONCURRENTLY` is used, statements generally take a `SHARE` lock type, meaning the operation has shared access to the resource. Without CONCURRENTLY, some operations will take an `EXCLUSIVE` lock on the table, meaning it's the only operation that can access the table for the length of time the lock is in effect.

Let's start with some of the places where you'll use it. Some of these operations you've not yet used here, but all examples will be covered.

- Adding an Index
- Dropping an Index
- Rebuilding an Index
- Adding a Partition
- Detaching a Partition

Once you've incorporated CONCURRENTLY on all your statements as your default way of working, you'll also want to know when it's not necessary and when not to use it.

Using CONCURRENTLY can make an operation like Creating an Index take twice as long. When adding an index to a table that's disconnected from any application or client queries, it's safe to create the Index without CONCURRENTLY, so that the creation is faster.